

Gartic Hands — Testing Guide

Who this is for: any teammate who needs to set up, run, and record testing for Gartic Hands the same way every time. Follow it top to bottom once; after that the cheat sheet is all you need.

Companion docs: `USER_GUIDE.md` (what a player sees), `CONTRIBUTING.md` (branching + commits), `DOCKER.md` (containers), `e2e-tests/README.md`, `component-tests/README.md`.

1. Cheat sheet

All commands run from `GarticHands/` unless the table says otherwise.

What	Command	Run from	Needs the app running?
Lint (client + load-test scenarios)	<code>npm run lint</code>	<code>GarticHands/</code>	No
Format check (non-mutating, informational)	<code>npm run format:check</code>	<code>GarticHands/</code>	No — see §5.1, currently reports pre-existing issues
Unit tests	<code>npm run test:unit -w @gartichands/client</code>	<code>GarticHands/</code>	No
Component tests	<code>npm run test:component -w @gartichands/client</code>	<code>GarticHands/</code>	No
End-to-end (Playwright)	<code>npm run test:e2e</code>	<code>GarticHands/</code>	No — Playwright starts server + client itself
E2E with the interactive runner	<code>npm run test:e2e:ui</code>	<code>GarticHands/</code>	No
One E2E spec	<code>npx playwright test --config=e2e-tests/playwright.config.ts e2e-tests/us3-join-with-code.spec.ts</code>	<code>GarticHands/</code>	No
Open the last E2E HTML report	<code>npx playwright show-report e2e-tests/playwright-report</code>	<code>GarticHands/</code>	No
Refresh visual-regression baselines	<code>npx playwright test --config=e2e-tests/playwright.config.ts e2e-tests/us21-visual-regression.spec.ts --update-snapshots</code>	<code>GarticHands/</code>	No
Load test (one scenario)	<code>k6 run load-testing/scenarios/five-player-game.js</code>	repo root	Yes — server on <code>:3000</code>
Coverage report	<code>npm run coverage -w @gartichands/client</code>	<code>GarticHands/</code>	No

Pass criteria for every automated suite: the command exits `0` and the Vitest / Playwright summary shows only `passed` (a `failed` count means a defect). ESLint **warnings** are allowed; ESLint **errors** are not. Anything else is a defect or an environment problem — see §8 Reporting defects. `format:check` is the one exception: it is informational (§5.1).

2. Purpose and scope

Gartic Hands is tested at five levels. Each level answers a different question, so none of them replaces another.

Level	Tool	Question it answers	Where it runs in CI
Lint + format	ESLint, Prettier	Does the code follow the agreed style and catch obvious mistakes?	Every PR to <code>main</code> (PR Lint)
Unit	Vitest	Do gesture detectors, canvas maths, prompt data and the REST client behave correctly in isolation?	Every PR to <code>main</code> (Unit Tests)
Component	Vitest + React Testing Library	Do pages, panels and UI primitives render and react correctly?	Every PR to <code>main</code> (Component Tests)

Level	Tool	Question it answers	Where it runs in CI
End-to-end (E2E)	Playwright (Chromium)	Can two real browsers create a room, play a full round and see the reveal? Plus visual regression, accessibility and UI-consistency checks.	Every push to <code>main</code> (<code>E2E Tests</code>)
Load	k6	Does the server stay fast and correct with several full rooms playing at once?	Every push to <code>main</code> (<code>Load Testing</code>)
Manual	You + a webcam	Does real hand-tracking, real audio, real multi-device play feel right?	Never — this is the part only a person can do (§7)

Out of scope for this guide: performance profiling of MediaPipe itself, and testing the read-only `legacy/` folder.

3. Prerequisites and test environment

Requirement	Version / notes	Check
Node.js	22.x locally (tested with v22.18.0). CI runs Node 24; both work.	<code>node -v</code>
npm	11.x (tested with 11.12.0)	<code>npm -v</code>
Git	any recent	<code>git --version</code>
Playwright browsers	Chromium only (the suite is configured for Chromium)	<code>npx playwright --version</code>
k6	only for load tests — https://grafana.com/docs/k6/latest/setup/install-k6/	<code>k6 version</code>
Webcam	only for manual drawing tests. Automated tests use a fake camera.	—
Free ports	<code>3000</code> (server), <code>5173</code> (client). Docker also uses <code>8080</code> .	<code>netstat -ano findstr :3000</code>
Browser for manual testing	Chrome or Edge (Chromium). Firefox/Safari are untested.	—

Automated tests never touch your real camera: Playwright launches Chromium with `--use-fake-device-for-media-stream` and drives the hand-tracking pipeline through a test-only seam (`window.__ghTestHooks.injectHandFrame`) that is enabled by a `gh:e2eHands` session-storage flag. It is never active during normal play.

4. Setup from a clean machine

```
git clone https://github.com/Monash-FIT3170/2026W2-GarticHands.git
cd 2026W2-GarticHands/GarticHands
npm install # installs client + server workspaces and git hooks
npx playwright install --with-deps # Chromium for the E2E suite (Windows: omit --with-deps if it errors)
```

Optional, for load tests only:

```
cd ../load-testing && npm install && cd ../GarticHands
```

Verify the setup in under two minutes:

```
npm run lint
npm run test:unit -w @gartichands/client
npm run test:component -w @gartichands/client
```

Lint may print warnings but must report `0 errors`; both test runs must show every file `passed`. If `npm install` fails with a lockfile error, do **not** delete `package-lock.json` — pull the latest `dev` first; the lockfile is shared across both workspaces and is committed.

Prefer Docker? `docker compose up -d --build` from `GarticHands/` runs the whole stack at `http://localhost:8080`. It is fine for manual testing, but the automated suites expect the dev servers on `:3000` / `:5173`, so run them from the host.

5. How to run each suite

5.1 Lint and format

```
npm run lint          # ESLint over client/ and load-testing/scenarios
npm run format:check  # Prettier, read-only
npm run format        # Prettier, rewrites files – run before committing
```

`npm run build` runs lint first (`prebuild`), so a lint failure also blocks the build.

Formatting is not currently enforced. `format:check` reports pre-existing style issues across ~90 files on `dev`, and the PR Lint workflow runs `npm run format` (the `--write` variant), which rewrites files inside the CI runner and always exits 0. Treat `format:check` as informational until the team decides to fix the baseline and switch CI to `format:check`. Do run `npm run format` on the files you touch.

5.2 Unit tests (`client/unit-tests/`)

```
npm run test:unit -w @gartichands/client
```

16 files. What they cover:

Area	Files
Canvas operations	<code>canvasDraw</code> , <code>canvasErase</code> , <code>canvasLocation</code>
Gesture detectors	<code>detectHandOnScreen</code> , <code>detectOpenPalm</code> , <code>detectPinch</code>
Gesture pipeline	<code>distance</code> , <code>gestureBuffer</code> , <code>gestureRecogniser</code> , <code>landmarkToCanvas</code>
Hand rendering	<code>drawHand</code> , <code>handConnection</code>
Camera hook	<code>useHandTracking</code>
Game data + API client	<code>prompts</code> , <code>revealChains</code> , <code>room</code>

Add a new file as `client/unit-tests/<thing>.test.ts`; Vitest picks it up automatically.

5.3 Component tests (`component-tests/`)

```
npm run test:component -w @gartichands/client
```

27 files: UI primitives (Button, Card, Badge, Avatar, Toast, CountdownTimer, PlayerList, RoundHeader, TopRightButtons, SettingsPanel), icons and logo, the drawing components (Canvas, DrawingStage, DrawingCameraCanvas, DrawingCameraInput, HandTracking), the pages (DrawPage, GuessingPage, InputPage, JoiningPage, Page) and the SettingsContext / `usePhaseAdvance` hooks. See `component-tests/README.md` for the relative-import rules and the `.tsx` gotcha before adding one.

5.4 End-to-end tests (`e2e-tests/`)

```
npm run test:e2e      # headless, list + HTML report
npm run test:e2e:ui   # Playwright UI runner – best for debugging one spec
npx playwright show-report e2e-tests/playwright-report
```

Playwright boots the server (`:3000`) and the Vite client (`:5173`) itself and reuses them if you already have `npm run dev` open. Runs are serial (`workers: 1`), with a 30 s per-test timeout, no retries, traces and screenshots kept only on failure. Artifacts land in `e2e-tests/test-results/` and `e2e-tests/playwright-report/`.

Spec	User story	What it proves
us1-create-room	US1	Host creates a room and appears in the lobby
us2-invite-code	US2	"Copy Room Code" puts a 6-character code on the clipboard
us3-join-with-code	US3	A second player joins with that code; both lobbies list both names
us4-ready-toggle	US4	Ready toggles are reflected on both screens
us5-start-game	US5	Host starts when everyone is ready; everyone lands on <code>/input</code>
us8-gesture-recognition	US8	<code>detectGesture</code> classifies labelled landmark fixtures with minimal error
us9-slideshow	US9	Reveal slideshow cycles every chain that has a drawing, manually and on auto-advance
us10-gesture-draw-stop	US10	A pinch draws a stroke; releasing it stops the stroke
us13-live-drawing	US13	The live stroke renders while drawing and survives submit
us15-recordings-replay	US15	"My Recordings" shows a playable replay per round
us16-host-start-guard	US16	Host cannot start from the UI while a player is not ready. Its second test is <code>test.fail</code> : the server's start endpoint does not enforce readiness yet (known bug)
us21-visual-regression	US21	Landing, host lobby and join page match the committed screenshots (≤ 1000 differing pixels)
us22-accessibility	US22	axe-core WCAG 2.0 A/AA scan; every control has an accessible name; "Host Game" is not clipped
us23-ui-consistency	US23	Logo on every route, consistent brand background, one primary heading per page

Three things that look wrong but are expected:

- **US22 marks two tests as `test.fail`** (landing page and host lobby). The current palette has known serious contrast violations; those tests are kept visible as a reminder and the run stays green *because* they fail. If one of them starts passing, Playwright reports it as a failure — that means the contrast was fixed and the `test.fail` wrapper should be removed.
- **US16 has one `test.fail` too.** The UI guard works, but `PATCH /rooms/:code/start` accepts the request even when a player is not ready. The test documents that server gap and will flip to a reported failure once it is fixed — then remove the wrapper.
- **US21 snapshots are per-OS.** Baselines exist for `linux` (CI) and `win32`. If you run on macOS the first run will fail with "missing snapshot"; add `--update-snapshots` once and commit the new baseline only if the screens genuinely changed.

5.5 Load tests (`load-testing/`)

The server must already be running on `:3000` (`npm run dev:server` from `GarticHands/`, or the full `npm run dev`). Then, from the **repository root**:

```
k6 run load-testing/scenarios/five-player-game.js
k6 run load-testing/scenarios/multiple-simultaneous-games.js
k6 run load-testing/scenarios/six-player-two-groups.js
k6 run load-testing/scenarios/late-player-join.js # see note
```

Every scenario shares the same thresholds: `http_req_failed < 1 %`, `http_req_duration p(95) < 500 ms`, `checks > 99 %`. k6 prints `✓/X` per threshold and exits non-zero on any breach.

Scenario	Shape
five-player-game	One host + four joiners complete a full prompt → draw → guess round
multiple-simultaneous-games	Two independent 3-player rooms run at once; checks they never leak into each other

Scenario	Shape
six-player-two-groups	Six players in one room, split into two "location" groups
late-player-join	Three players start; two join mid-round. The scenario asserts that late joins are accepted and flagged <code>joinedMidRound</code> , which is what the server does today. It is still commented out in CI behind an older note that expected late joins to be rejected. Run it locally; if it passes, the CI step can be re-enabled.

5.6 Git hooks (run automatically)

Installed by `npm install` via Husky. They are part of the test surface because they block commits and pushes:

Hook	What it runs	Fails when
pre-commit	<code>npm run check:branch</code>	Branch name is not <code>feature/...</code> , <code>bugfix/...</code> , <code>docs/...</code> , <code>refactor/...</code> , <code>test/...</code> or <code>chore/...</code> (kebab-case)
commit-msg	<code>commitlint</code>	Message is not <code><type>(<scope>): <summary></code> with type in <code>feat fix docs refactor test chore</code> — the scope is required
pre-push	<code>npm run check:branch</code>	Same as pre-commit

Note the mismatch: branch prefixes are `feature / bugfix`, commit types are `feat / fix`.

6. What CI runs, and when

Workflow	Trigger	Runs
PR Lint	pull request → <code>main</code>	<code>npm run lint</code> , then <code>npm run format</code> (Prettier <code>--write</code> — rewrites in the runner, so it never fails; see §5.1)
Unit Tests	pull request → <code>main</code>	<code>npm run test:unit</code>
Component Tests	pull request → <code>main</code>	<code>npm run test:component</code>
E2E Tests	push → <code>main</code> (after merge)	<code>npm run test:e2e</code> ; uploads <code>playwright-report</code> as an artifact for 14 days
Load Testing	push → <code>main</code> (after merge)	Installs k6, starts the server, runs 3 of the 4 scenarios

PRs to `dev` do **not** trigger CI. Before opening a `dev` → `main` promotion PR, run the full cheat-sheet locally so the E2E and load jobs (which only run post-merge) do not surprise the team. `main` additionally requires **two approving reviews** and resolved threads.

To download an E2E report from CI: open the failed run on GitHub → *Artifacts* → `playwright-report` → `unzip` → `npx playwright show-report <folder>`.

7. Manual test areas

Automated tests use a fake camera and synthetic hand landmarks, so the areas below must be checked by a person before a demo or release. Run them in **two browser windows on one machine** (host in one, joiner in the other) and, if possible, once more on **two separate laptops** on the same Wi-Fi.

Legend for *Expected*: what must be true for the step to **pass**. Anything else is a defect.

7.1 Lobby and joining

#	Step	Expected
L1	Landing → enter name → Host Game	<code>/host</code> lobby, your name with a Host badge, six-character room code visible
L2	Copy Room Code	Toast "Invite code copied!" (host) or "Room code copied!" (joiner); clipboard holds the code

#	Step	Expected
L3	Second window → Join Room → name + code → Join Game	Joiner lands on <code>/joined/<code></code> ; both windows list both players
L4	Wrong or expired code	Clear error on the join page; no navigation
L5	Joiner presses Ready Up / Ready to toggle	Badge flips on both screens within ~1 s; the "n/n ready" counter updates
L6	Host presses Start Game while someone is not ready	Start is refused (button disabled or message); nothing navigates
L7	All ready → Start Game	Every window moves to the prompt page at the same time
L8	Leave Room as host	Host role passes to the longest-standing remaining player; room continues
L9	Join a room whose game already started	Joiner sees the toast "You joined mid-round — you'll play from the next round!" and a "Round N is still being played" screen; the round is not blocked by them and they play from the next round

7.2 Prompt phase

#	Step	Expected
P1	Type a prompt → Submit	Waiting state; the timer keeps counting
P2	Let the 60 s timer run out without submitting	Whatever is typed is auto-submitted; if empty, a random fallback prompt is used; the phase still advances
P3	All players submit early	Everyone moves to the draw page immediately

7.3 Drawing with the webcam (the part no automation covers)

#	Step	Expected
D1	First visit to the draw page	Browser asks for camera permission once; camera preview appears, status pill reads NO_HAND until a hand is shown
D2	Show a relaxed hand (fingers loosely curled, not pinching, not flat)	Pill changes to HAND_PRESENT ; a cursor follows the index fingertip on the canvas
D3	Pinch index finger + thumb and move	Pill reads PINCH ; a continuous stroke follows the fingertip; releasing the pinch ends the stroke without a tail
D4	Open palm (all four fingers up)	Pill reads OPEN_PALM ; erases where the hand is
D5	Switch layout tabs Camera + Canvas / Draw on Camera / Camera + Overlay + Canvas	Drawing continues; nothing is lost when switching
D6	Before starting, in the lobby: gear icon → Gesture sensitivity: High	Lighter pinch registers on the draw page; Low needs a firmer pinch. Setting survives a page reload. (The gear is only available on the landing, join and lobby pages, not while drawing)
D7	Before starting, in the lobby: gear icon → Stroke smoothing: Strong	Wobbly lines visibly smoother; Light follows the hand more literally
D8	Draw in poor light / hand partly out of frame	Tracking may drop to NO_HAND ; strokes should stop, not scribble
D9	Submit Drawing or let the timer expire	The current canvas (even if blank) is submitted; page moves to guessing

7.4 Guessing and reveal

#	Step	Expected
G1	Guess page	You see another player's drawing (never your own with 2+ players), "Drawn by ", a countdown
G2	Submit a guess / let the timer expire	Empty guess is submitted on timeout; page moves to reveal when all are in

#	Step	Expected
R1	Reveal tab	One card per player: " wrote" → drawing (or "No drawing submitted") → " guessed", each guess attached to the drawing it was made about
R2	Slideshow tab	Steps through every chain that has a drawing (chains with no drawing are skipped); auto-advance every 4 s when there are 2+ slides; Prev / Pause / Next work
R3	My Recordings (N) tab	One replay per round you drew; plays back your strokes
R4	Host → Play Round N+1	Round badge increments (up to 4); everyone returns to the prompt page; non-hosts see a waiting message until then
R5	After round 4, host → Back to Lobby	Everyone returns to the lobby; recordings are cleared; no stuck "waiting" state

7.5 Settings and accessibility

#	Step	Expected
S1	Gear icon (top-right) → Colour vision: Deuteranopia / Protanopia / Tritanopia	Palette changes immediately and stays after reload
S2	Volume and book icons (top-right)	Currently do nothing — this is known. Record it, do not file a defect unless a click causes an error
S3	Keyboard only: Tab through landing, join, lobby	Every control is reachable and visibly focused
S4	Window at 1024 px wide and at 1920 px	No clipped buttons, no horizontal scrollbar

7.6 Resilience

#	Step	Expected
X1	Close a joiner's tab mid-round	Within ~30 s the player is dropped; the round advances once everyone <i>remaining</i> has submitted
X2	Refresh the browser mid-game	Known limitation: the player is bounced to / and removed from the room. Record it, do not file a new defect unless behaviour differs
X3	Restart the server mid-game	Known limitation: rooms are in memory; every client will error. Same as X2

8. Reporting defects

File defects at <https://github.com/Monash-FIT3170/2026W2-GarticHands/issues> using the template below. One defect per issue. Attach evidence **before** asking anyone to reproduce.

```

**Title:** <area>: <one-line symptom>                e.g. draw: pinch stroke keeps a tail after release

**Severity:** S1 blocker | S2 major | S3 minor | S4 cosmetic
- S1: game cannot be played / data lost / crash
- S2: a feature does not work but there is a workaround
- S3: works but wrong, ugly, or slow
- S4: typo / spacing / colour

**Environment**
- Branch + commit: `git rev-parse --short HEAD`
- How it was run: `npm run dev` | Docker | CI run <link>
- OS / browser + version:
- Camera (manual tests only): built-in / external, lighting
- Players: how many windows / machines

**Steps to reproduce**
1. ...
2. ...
3. ...

**Expected** (quote the row from TESTING_GUIDE.md §7 if it applies, e.g. "D3")
**Actual**

**Evidence** (at least one)
- Screenshot / screen recording
- Playwright: `e2e-tests/test-results/<spec>/trace.zip` (open with `npx playwright show-trace`)
- Console output / server log excerpt
- k6 summary block

**Reproducibility:** always | sometimes (x of y tries) | once

```

Recording a test session (so the team can see what was covered, not just what broke):

```

## Test session – <date> – <tester> – <branch@sha>
Suites: lint  | unit  (n passed) | component  (n passed) | e2e / | load //skipped
Manual areas run: 7.1  7.2  7.3  (D3, D6) 7.4  7.5  skipped 7.6  skipped
Defects filed: #<n>, #<n>
Notes: <anything odd that did not become a defect>

```

Paste session records into the team's user-story tracking sheet (linked from [e2e-tests/README.md](#)) or the PR description that the session was testing.

9. Troubleshooting

Symptom	Likely cause	Fix
Error: browserType.launch: Executable doesn't exist	Playwright browsers not installed	<code>npx playwright install</code>
E2E hangs on the first test	Port 3000 or 5173 already in use by something else	Stop it, or run <code>npm run dev</code> yourself so Playwright reuses it
<code>us21</code> fails with "missing snapshot"	New OS without a baseline	Run once with <code>--update-snapshots</code> ; only commit if intended
<code>us22</code> shows 2 and <code>us16</code> shows 1 "expected failure"	Expected (see §5.4) — the run is still green	Nothing
Vitest: Failed to start forks worker / Timeout waiting for worker to respond, some files never run	Machine under load (dev servers, Playwright or another suite running at the same time)	Re-run the suite on its own; do not run unit and component suites concurrently

Symptom	Likely cause	Fix
Component test: "Cannot find module" for a file that exists	Test file moved outside <code>component-tests/</code> , or JSX in a <code>.ts</code> file	Keep tests in <code>component-tests/</code> , use <code>.tsx</code>
k6 <code>late-player-join</code> fails a <code>checks</code> threshold	Server no longer accepts or flags mid-round joins the way the scenario expects	Real regression — file it; the scenario matches current intended behaviour
Commit rejected: "Invalid branch name"	Branch prefix is <code>feat/</code> or <code>fix/</code>	Rename to <code>feature/...</code> or <code>bugfix/...</code>
Commit rejected: "scope may not be empty"	Commit message lacks <code>(scope)</code>	e.g. <code>test(e2e): add late-join spec</code>
Camera preview black in manual test	Another app holds the webcam, or permission denied	Close the other app; reset site permissions in the browser